Unit 12 – Approximation in Value Space: Neural Networks, Deep Q-Learning and LSTD

4SC000 Optimal Control and Reinforcement Learning

Duarte Antunes

Slides · Jupyter notebooks

Where this unit sits in the course

Units 10 and 11 built the machinery of reinforcement learning on a promise that was never going to hold: that the cost-to-go can be stored in a table. Q-learning keeps one number per state-action pair; policy iteration keeps one per state; SARSA keeps a table too. Unit 10 was explicit that this is the **curse of dimensionality** untouched – the table has nm entries, and for a continuous state space discretized finely enough that is hopeless.

This unit removes the table. In its place goes a **parameterised approximation** $\tilde{J}(x, r)$ or $\tilde{Q}(x, u, r)$, with far fewer parameters than there are states, and everything else in Units 10 and 11 is rebuilt on top of it. That single change is what makes the methods applicable to problems with continuous states, and it is what the deep reinforcement learning of the last decade consists of.

There is a price, and it is worth naming at the outset rather than at the end. Almost every guarantee from Units 10 and 11 is lost. Those proofs rested on the fact that updating one table entry leaves all the others alone, so that the Bellman operator is a contraction in the maximum norm. With an approximation there are no independent entries: changing a weight changes the predicted value at *every* state at once, the contraction argument collapses, and the algorithms of this unit are supported by experiment rather than by theorems. The one exception – linear features, where something exact can still be said – is the subject of the last section.

The unit has three parts, following the deck. First, what an approximation architecture is and how it is trained, which turns out to be a **regression** problem and nothing more exotic. Second, **deep Q-learning**: Unit 10's algorithm with a neural network in place of the table, plus the two devices that make it work. Third, approximation inside **policy iteration**, which is where the distinction between a stochastic gradient and a **semi-gradient** appears, and where linear features let us compute a fixed point exactly by **LSTD**.

Approximation architectures

Recall the approach of Unit 8, which is also the approach of everything since. Act greedily with respect to an approximate cost-to-go,

$$u_k = \arg \min_{u_k \in U_k(x_k)} \mathbb{E}[g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}(f_k(x_k, u_k, w_k))],$$

where $\tilde{J}_{k+1}(x_{k+1}, r_{k+1})$ is a parameterised approximation of the optimal cost-to-go. For the rest of this unit the stage subscript is written k rather than $k+1$, purely to keep the formulas readable.

Two questions then arise, and they are independent of each other. What family of functions should $\tilde{J}_k(\cdot, r_k)$ be drawn from – the **architecture**? And how are the parameters r_k chosen – the **training**?

The training question has the same answer whatever the architecture. Given a set of q samples β_s of the cost-to-go, ideally close to the optimal $J^\star(x_k)$, at states x_k^s, choose

$$\min_{r_k} \sum_{s=1}^{q} \left(\tilde{J}_k(x_k^s, r_k) - \beta_s\right)^2$$

for a large number q of states. This is least-squares fitting, and it is the entire training procedure. Where the targets β_s come from is a separate matter, and is what distinguishes the algorithms later in the unit: a Monte-Carlo return, a TD target, or a Bellman backup.

Linear architectures

The simplest useful architecture is linear in the parameters:

$$\tilde{J}_k(x_k, r_k) = \sum_{\ell=1}^{m_k} r_{\ell,k}\phi_{\ell,k}(x_k) = r_k^\top \phi_k(x_k).$$

Fig. 1 shows the arrangement. The state passes through a **feature extraction** stage producing a vector of features $\phi_{\ell,k}(x_k)$, and the approximation is a linear combination of them.

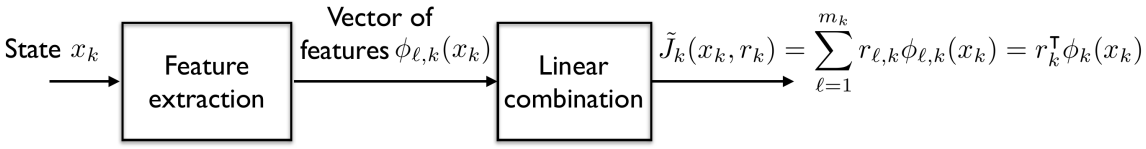


Fig. 1: A linear architecture. The state is mapped to a vector of features by a fixed, hand-designed map ϕ, and the approximation is a weighted sum of those features. Only the weights r_k are learned; the features are not.

The word "linear" is doing less work than it looks. The approximation is linear in the *parameters*, not in the state: the features may be arbitrarily nonlinear functions of x_k. With the monomials

$$\phi_{0,k}(x_k) = 1, \quad \phi_{1,k}(x_k) = x_k, \quad \phi_{2,k}(x_k) = x_k^2, \quad \phi_{3,k}(x_k) = x_k^3, \ \ldots$$

one can in principle approximate any function $J^\star(x_k)$ arbitrarily closely. So a linear architecture can be very powerful – provided the adequate features are chosen.

Its major advantage over everything that follows is that **training is trivial**. The objective is a quadratic in r_k; setting its derivative to zero gives a closed form,

$$r_k = \Big[\sum_{s=1}^{q} \phi_k(x_k^s)\,\phi_k(x_k^s)^\top\Big]^{-1} \sum_{s=1}^{q} \phi_k(x_k^s)\,\beta_s,$$

a single linear solve with no iteration, no step size, and no local minima. That property returns at the end of the unit, where it is what makes LSTD possible.

The disadvantage is the proviso. Somebody has to choose the features, and on a problem where one does not already understand the structure of the cost-to-go, that is the whole difficulty rather than a preliminary.

Nonlinear architectures

Allowing $\tilde{J}_k(x_k, r_k)$ to depend nonlinearly on r_k buys expressiveness and costs the closed form. The objective

$$\min_{r_k} \sum_{s=1}^{q} \left(\tilde{J}_k(x_k^s, r_k) - \beta_s\right)^2$$

is no longer quadratic, so in general one must pick a numerical method, typically a gradient method, and since the cost function is no longer convex one might get stuck in local minima.

Still, the objective has a special structure – it is a **sum over samples** – and that structure allows an *incremental* method, where the parameters are updated after each simulation rather than after sweeping all of them. Compare

$$\textbf{Normal gradient} \qquad \hat{r}_k^{\ell+1} = \hat{r}_k^{\ell} - \gamma \sum_{s=1}^{q} \frac{\partial}{\partial r_k} \left(\tilde{J}_k(x_k^s, r_k) - \beta_s\right)^2 \Big|_{r_k = \hat{r}_k^{\ell}}$$

with

$$\textbf{Incremental gradient} \qquad \hat{r}_k^{\ell+1} = \hat{r}_k^{\ell} - \gamma \frac{\partial}{\partial r_k} \left(\tilde{J}_k(x_k^s, r_k) - \beta_s\right)^2 \Big|_{r_k = \hat{r}_k^{\ell}},$$

where the second uses one sample s in place of the whole sum. Neural networks, a special case of nonlinear architectures, are trained by exactly this incremental method – under the name **backpropagation**, which is not a different algorithm but an efficient way of evaluating that derivative.

Quick exercise: if linear architectures can approximate anything, why use anything else?

With monomial features a linear architecture approximates any continuous function arbitrarily closely, and its training is a single linear solve with no local minima. A neural network is harder to train, has no closed form, and can get stuck. What does the network give in return?

Solution

It chooses its own features.

The approximation guarantee for monomials is real but is a statement about one-dimensional problems, or about fixing the dimension and letting the degree grow. In n dimensions the number of monomials of degree at most d is $\binom{n+d}{d}$, which for a state of dimension 20 and degree 4 is already above ten thousand – and the right features for a control problem are rarely monomials anyway. The curse of dimensionality has not been removed by writing $r^\top \phi(x)$; it has been moved into the length of ϕ.

What makes a linear architecture work in practice is a *short* feature vector that happens to capture the structure of the problem, and finding one requires knowing the answer in advance. That is the honest reading of "as long as the adequate features are chosen".

A deep network's layers, by contrast, can learn to extract features from the output of previous layers, so the designer is released from having to handcraft them. That is the trade: a harder optimisation problem, in exchange for not having to solve the feature-design problem by hand. For the lunar lander of this unit, nobody knows what the right features for the cost-to-go are, and the network finds something that works from an eight-dimensional raw state.

One caveat worth keeping. The exchange is not free in theory – a linear architecture has a unique minimum and a network does not – so when good features *are* known, the linear architecture remains the better choice. This is why the last section of this unit, on LSTD, is not a historical footnote.

Neural networks

A neural network is a particular nonlinear architecture, built by alternating linear maps with a scalar nonlinearity.

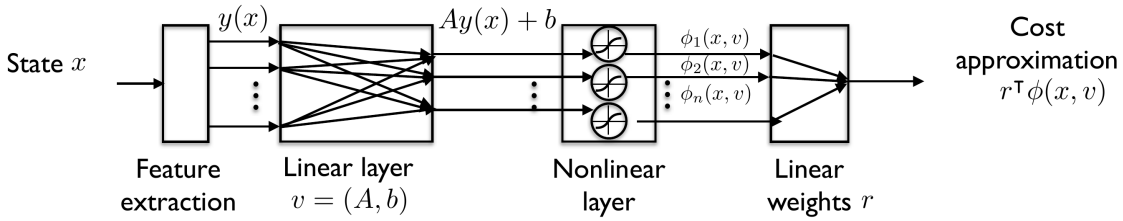


Fig. 2: The typical structure. The state is mapped to features $y(x)$, a linear layer with parameters $v = (A, b)$ produces $Ay(x) + b$, a nonlinear layer applies a scalar function σ componentwise, and a final linear layer with weights r combines the results into the cost approximation.

Written out, the structure of Fig. 2 is

$$\tilde{J}(x, v, r) = \sum_{\ell=1}^{m} r_\ell \, \sigma((Ay(x) + b)_\ell),$$

the subscript ℓ denoting component ℓ of the vector inside. The parameters are $v = (A, b)$ and r together.

The nonlinearity σ is a fixed scalar function applied componentwise; Fig. 3 shows the two standard choices.

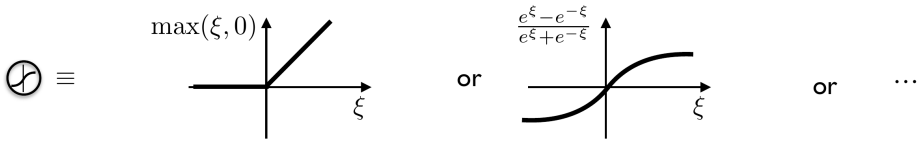


Fig. 3: Two common choices of the nonlinearity σ: the rectifier $\max(\xi, 0)$ on the left, and the hyperbolic tangent $(e^\xi - e^{-\xi})/(e^\xi + e^{-\xi})$ on the right.

This structure is relatively easy to train – in contrast with the multi-layer deep networks below – provided the right features $y(x)$ are chosen, which in turn means somebody must still handcraft them.

One consequence of the rectifier worth carrying into the examples: a network built from $\max(\xi, 0)$ is **piecewise linear**. Every curve it draws is a polyline, and any smoothness one sees is a large number of short segments. It is also why such a network extrapolates badly beyond the range of its training data – outside that range it simply continues its last linear piece, which is visible at the edges of the fits below.

Universal approximation

The structure of Fig. 2 is a **universal approximator**: it can approximate arbitrarily closely any continuous function $J : \mathbb{R}^n \to \mathbb{R}$. The intuition, for $J : \mathbb{R} \to \mathbb{R}$, is that it can build a pulse, as in Fig. 4.

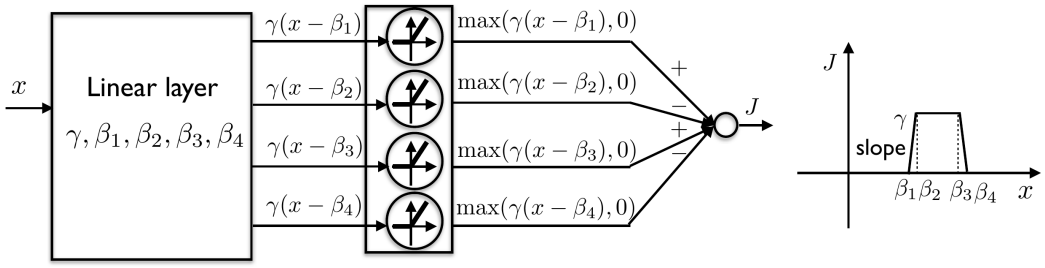


Fig. 4: Building a pulse from four rectifiers. A linear layer forms $\gamma(x - \beta_i)$ for four offsets; the nonlinear layer rectifies each; and the output layer adds the first and third while subtracting the second and fourth. The result is a trapezoid of slope γ rising between β_1, β_2 and falling between β_3, β_4.

Take four offsets $\beta_1 < \beta_2 < \beta_3 < \beta_4$ and a slope γ. The combination

$$\max(\gamma(x - \beta_1), 0) - \max(\gamma(x - \beta_2), 0) + \max(\gamma(x - \beta_3), 0) - \max(\gamma(x - \beta_4), 0)$$

is zero below β_1, rises with slope γ to β_2, is flat between β_2 and β_3, falls back between β_3 and β_4, and is zero afterwards – a trapezoidal pulse of adjustable position, width and height, built from four units. And a continuous function can be approximated by a sum of pulses. Since each pulse costs four units, a network with enough units can approximate any continuous function.

The argument is worth understanding and worth not over-reading. It says that the *architecture* is rich enough; it says nothing about whether the training procedure will find the right parameters, nor how many units "enough" is. In high dimensions the number of pulses needed to tile the space grows exponentially, so universality is a reassurance about expressiveness, not a solution to the curse of dimensionality.

Deep networks

Stacking the construction gives a deep network, Fig. 5: several linear and nonlinear layers between input and output.

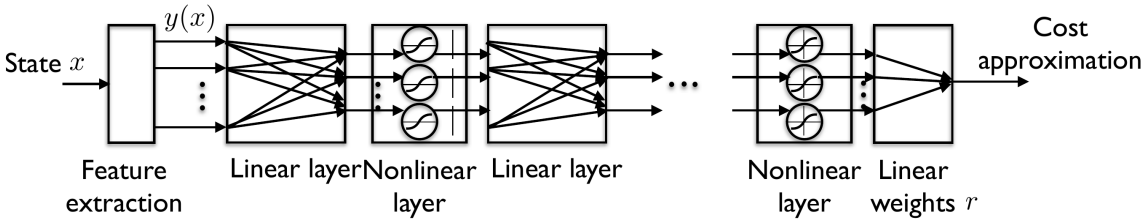


Fig. 5: A deep network: feature extraction, then an alternating sequence of linear and nonlinear layers, ending in a linear combination with weights r. Each layer's output is the next layer's input.

Three observations.

Deep networks are responsible for the recent achievements in computer vision, and much of that success is expected in other fields.

They are much harder to train, and much more insight is needed to design the structure.

But in some cases some of the layers learn how to **extract features** from the input of previous layers, releasing the designer from having to handcraft them. That is the property that matters for this course: it is the answer to the objection raised against linear architectures above.

For more, the standard reference is *Deep learning*, Ian Goodfellow, Yoshua Bengio and Aaron Courville, MIT Press, 2016.

Training is regression

Before putting a network inside a reinforcement learning algorithm it is worth being precise about what fitting one means, because the answer determines what the algorithms later can and cannot achieve.

What least-squares regression computes

Consider a regressor that maps a state x to an estimate $\hat{J}$, where the pair (x, J) follows a joint probability distribution. The regression problem is

$$\min_{\hat{J}} \ \mathbb{E}\big[(J(x) - \hat{J})^2\big],$$

and its solution is

$$\hat{J} = \mathbb{E}[J \mid x],$$

the conditional mean. Fig. 6 shows the situation: a cloud of probability mass over (x, J) pairs, and the curve the regression picks out through it.

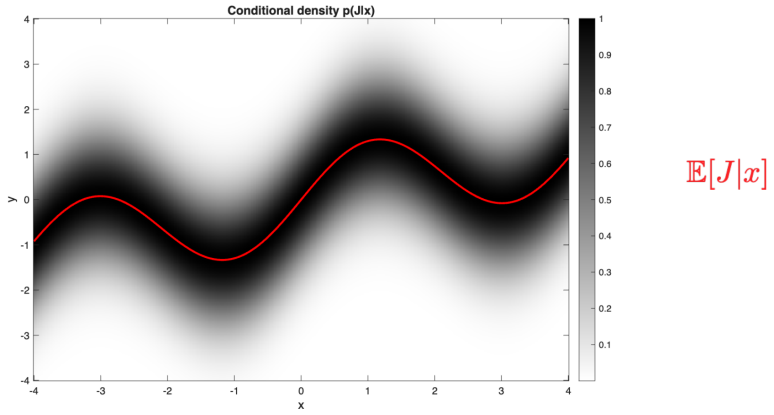


Fig. 6: The conditional density $p(J \mid x)$ as a grey cloud, with the conditional mean $\mathbb{E}[J \mid x]$ in red. Least-squares regression computes the red curve, whatever the shape of the cloud around it.

This is the single most useful fact in the section. Fitting a squared error does not recover the function that generated the data; it recovers the **conditional expectation**. When the data are noisy observations of a deterministic function, those coincide, and all is well. When the targets are themselves random – as every Monte-Carlo return and every sampled Bellman backup is – the regression is estimating an expectation, which is precisely what the cost-to-go is defined to be. The algorithms later in the unit rely on this silently and constantly.

Fitting with a quadratic loss

In practice the distribution is not available, only samples. Given samples (x_i, J_i) of the joint distribution, a network with parameters θ is fitted by

$$\min_{\theta} \ \frac{1}{N} \sum_{i=1}^{N} \big(J_i - \hat{J}(x_i; \theta)\big)^2.$$

Fig. 7 shows what that recovers from a finite sample.

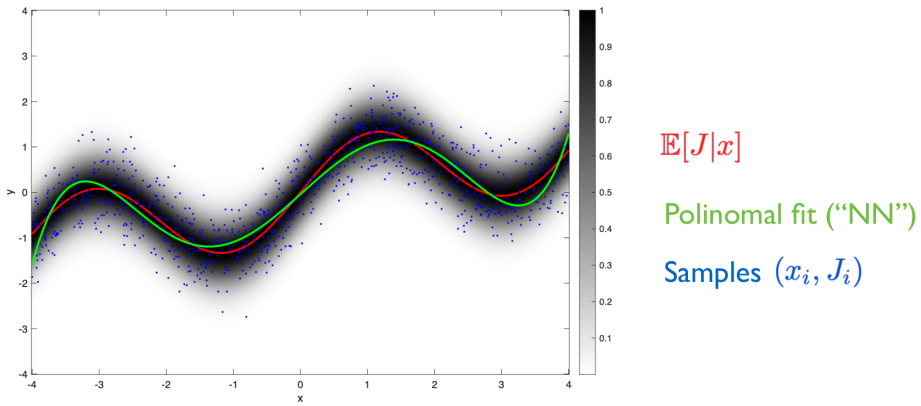


Fig. 7: Samples drawn from the cloud (blue), the conditional mean (red), and a fit (green). As the architecture becomes more expressive and more samples are acquired, the best fit approaches $\mathbb{E}[J \mid x]$.

Fig. 8 shows the same experiment with an actual network: two hidden layers of 64 rectified units, trained for 400 epochs on 300 noisy samples of $\sin(2\pi x) + 0.3x$.

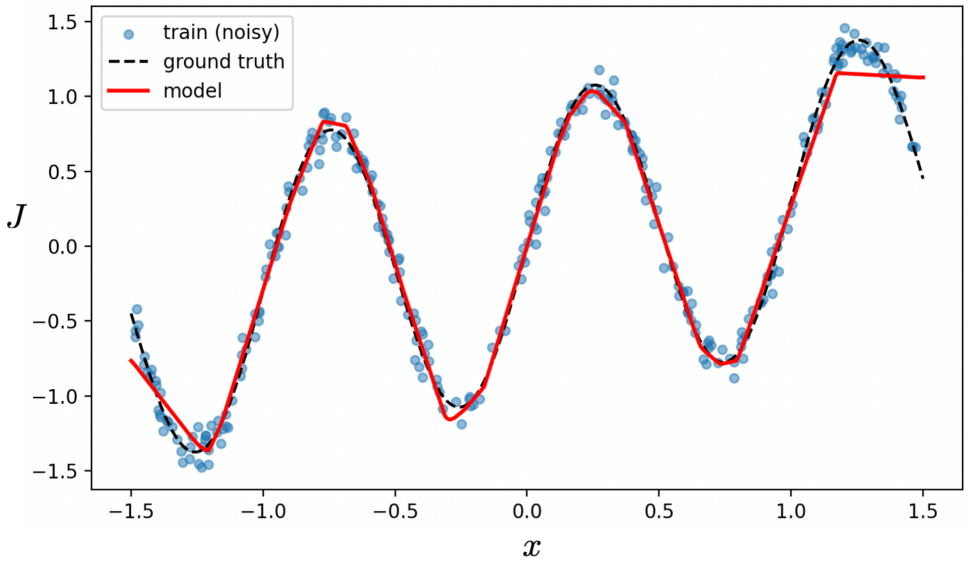


Fig. 8: A real fit. Blue dots are the noisy training samples, the dashed black curve is the function they were generated from, and the red curve is the trained network. The flattening of the red curve at the right-hand edge is the piecewise-linear extrapolation discussed above.

Two features of that picture are worth stating, because both recur in the reinforcement learning examples.

The fit passes through the middle of the cloud, not through the points. That is the conditional mean again, and it is the correct behaviour rather than a failure to fit. A useful quantitative check is the **noise floor**: with additive noise of standard deviation σ_n, no fit can achieve a mean squared error below σ_n^2 against the noisy targets. In the example $\sigma_n = 0.08$, so the floor is 0.0064, and the achieved training loss of about 0.013 says the network has captured most but not all of the structure. A loss well *below* the floor would be a warning that the network is fitting noise.

The fit degrades at the edges of the data. Nothing constrains a network outside the range it was trained on, and a rectified network extrapolates linearly. In a control setting this matters more than it does in curve fitting: a policy that is greedy with respect to a cost-to-go approximation will happily drive the state into a

region where the approximation is an extrapolation, and the approximation will confidently return a number with nothing behind it.

`A_12_NNRegression.ipynb` runs this fit and prints the noise floor alongside the loss.

Stochastic gradient

Following a gradient descent method would be very expensive when N is very large, because every step of

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \left(J_i - \hat{J}(x_i;\theta)\right)^2$$

requires a pass over all N samples. So the training of neural networks uses the **stochastic gradient** method, in which the full gradient is replaced by the gradient of one squared sample error,

$$\theta_{k+1} = \theta_k - \gamma_k \nabla_\theta \left(J_i - \hat{J}(x_i;\theta)\right)^2,$$

with i drawn at random. This simple gradient can be computed efficiently for neural networks by the back propagation algorithm.

Three things recommend it. It converges under mild assumptions – the Robbins-Monro conditions of Unit 10, which is not a coincidence, since Q-learning is itself a stochastic approximation scheme. It is dramatically cheaper per step. And the randomness allows it to escape local minima that would trap a deterministic gradient method.

The deck compares the two on the regression example above:

Table 1: Stochastic against full gradient on the regression of Fig. 8.

	Time	Loss	Training epochs
Stochastic gradient	0.871 s	0.013107	400
Full gradient	16.950 s	0.011145	120 000

⚠ Two corrections to that table

The epoch count printed on the slide is 12 000; the script computes `epochs*300`, which is 120 000. Running both settles it: at 12 000 epochs plain gradient descent reaches a training loss of 0.2796, nowhere near converged, while 120 000 reproduces the slide's own figure of 0.011145 exactly. The losses and the 400 epochs are right; only the epoch count for the full gradient was wrong, and it is corrected above.

More substantively, the comparison varies **two** things at once. The stochastic run uses Adam; the full-gradient run uses plain gradient descent with a fixed step size. Repeating the full-gradient run with Adam, at an equal number of parameter updates, makes it about as fast as the stochastic one on this problem – so the twenty-fold gap in the table is mostly the optimiser, not the batch size.

That does not overturn the argument for stochastic gradient; it locates it. With 255 training points the full gradient is cheap. The cost of one full-batch step grows linearly in N while the cost of one minibatch step does not, so the argument bites when N is large – as it is in the deep Q-learning below, where the data set is a replay buffer of 50 000 transitions that is still growing.

`A_12_StochasticVsFullGradient.ipynb` runs all of these.

Quick exercise: what is a 'sample of the cost-to-go'?

The training objective needs targets β_s that are "samples of the cost-to-go, ideally close to the optimal $J^\star(x_k)$". But $J^\star$ is exactly the thing being computed – if it were available there would be nothing to fit. Where do the targets come from?

Solution

From the simulator, and the whole of Units 10 and 11 is a catalogue of ways to produce them. Three appear in this unit.

A Monte-Carlo return. Run the policy from x_k^s and add up what it actually costs. This is an unbiased sample of $J_\mu(x_k^s)$ – of the *current policy's* cost, not of $J^\star$ – which is why it belongs to policy evaluation inside policy iteration, and is the subject of the last section.

A TD target, $g(x_k, \mu(x_k)) + \alpha\hat{J}(x_{k+1};\theta)$. Cheaper, available after one step rather than a whole trajectory, and biased, because it uses the current estimate at x_{k+1} rather than the true value. This is the Unit 11 trade-off, unchanged.

A Bellman backup, $g(x_k, u_k) + \alpha \min_u \hat{Q}(x_{k+1}, u; \theta)$. This targets $J^\star$ rather than J_μ, because of the minimisation, and it is what deep Q-learning fits.

The thing to notice is that in the second and third cases the target **contains the parameters being fitted**. The regression is chasing a target that moves whenever the fit moves. Nothing in the regression theory above covers that situation: the clean statement "least squares computes $\mathbb{E}[J \mid x]$" assumed a fixed joint distribution of (x, J), and here the distribution of J depends on θ.

That single observation is the source of both of the remaining sections. Deep Q-learning's two special features exist to slow the target down, and the distinction between a stochastic gradient and a semi-gradient is precisely a question of whether one differentiates through the target or not.

Deep Q-learning

To extend Q-learning to continuous state spaces – still with a discrete set of actions – approximate the Q-factors with a neural network. The fit with the incremental training of the previous section is exact: **each time a cost is received, one gradient step is taken** – the loop drawn in Fig. 9.

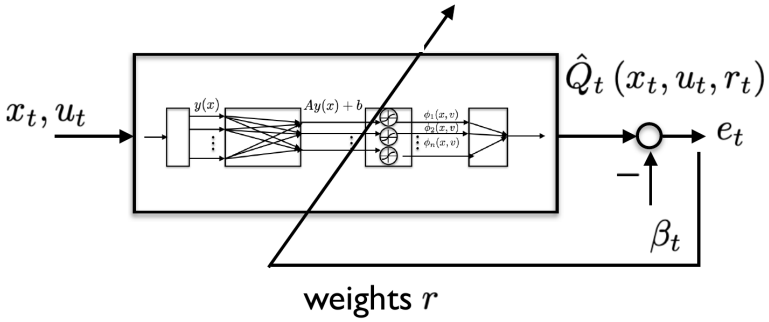


Fig. 9: Q-learning with a network. The pair (x_t, u_t) enters the network, which outputs $\hat{Q}_t(x_t, u_t, r_t)$; the target β_t is subtracted to give the error e_t, and the error is propagated back into the weights r.

In formulas, the weights move by

$$\hat{r}_{t+1} = \hat{r}_t - \gamma \frac{\partial}{\partial r}(\hat{Q}_t(x_t, u_t, r) - \beta^t)^2\Big|_{r=\hat{r}_t},$$

with the target

$$\beta^t = g(x_t, u_t) + \min_{v \in U(x_{t+1})} \hat{Q}_t(x_{t+1}, v, \hat{r}_t).$$

This is a natural extension of the tabular case, which can be rewritten in the same shape:

$$Q_{t+1}(x_t, u_t) = Q_t(x_t, u_t) - \gamma^t (Q_t(x_t, u_t) - \beta^t), \qquad \beta^t = g(x_t, u_t) + \min_{v \in U(x_{t+1})} Q_t(x_{t+1}, v).$$

Reading the two side by side is the clearest way to see what has changed. The tabular update moves *one entry* towards its target. The network update moves *every weight*, and therefore changes the predicted Q-factor at every state-action pair at once. The tabular convergence proof of Unit 10 used the first property and does not survive the second.

The two special features

Deep Q-learning is this idea plus two devices. Both exist for the reason the last exercise identified: the target contains the parameters being fitted.

The main idea, stated as on the deck: approximate the Q-function with a neural network to handle large or continuous state spaces,

$$Q^\star(x_k, u_k) \approx \hat{Q}(x_k, u_k, \theta),$$

and learn θ by using the error to train the network,

$$L(\theta) = \Big(g(x_k, u_k) + \alpha \min_u \hat{Q}(x_{k+1}, u, \theta) - \hat{Q}(x_k, u_k, \theta) \Big)^2.$$

Experience replay. Store transitions $(x_t, u_t, g(x_t, u_t), x_{t+1})$ in a replay buffer and train on random minibatches drawn from it, rather than on the transitions in the order they arrive. Consecutive transitions from one trajectory are strongly correlated – they come from nearby states under nearly the same policy – and fitting a regression to a correlated stream is the one thing least-squares theory is least equipped for. Sampling at random from a large buffer breaks the correlation and stabilises learning. It also means each transition is used many times, which is why deep Q-learning is comparatively sample efficient.

A target network. Keep a second copy of the network, $\bar{\theta}$, updated only every L steps, and compute the target with it. Without it, every gradient step changes the target of the very regression it is solving, and the two chase each other. Freezing the target for a thousand steps turns the problem back into an ordinary regression against a fixed target, for a thousand steps at a time.

`A_12_LunarLanderDQN.ipynb` reruns the algorithm with each device disabled – the buffer shrunk to one minibatch, and the target network copied every step – which is the quickest way to see that this is what they are for.

The algorithm

> **Deep Q-learning**
>
> Initialize the replay buffer $\mathcal{D}$ to capacity N, the action-value network $Q(x, u, \theta)$ and the target network $Q(x, u, \bar{\theta})$ with random weights $\theta = \bar{\theta}$.
> For each episode:
> A) Initialize the state x_0.
> B) For each time step k:

1. Select $u_k = \arg\min_u Q(x_k, u, \theta)$ with probability $1 - \epsilon$, otherwise a random action.
2. Store the transition $(x_k, u_k, g(x_k, u_k), x_{k+1})$ in $\mathcal{D}$.
3. Sample a minibatch of M transitions from $\mathcal{D}$.
4. Compute the target for each sample:

$$z_k = \begin{cases} g(x_k, u_k) + g(x_{k+1}, \cdot), & \text{if } x_{k+1} \text{ is terminal} \\ g(x_k, u_k) + \alpha \min_u Q(x_{k+1}, u; \bar{\theta}), & \text{otherwise.} \end{cases}$$

5. Perform a gradient descent step on the loss – a back propagation step in the network.
6. Every L steps, update the target network: $\bar{\theta} = \theta$.

Note which network appears where. The action is selected with θ, the gradient step moves θ, and only the *target* in step 4 uses $\bar{\theta}$. Step 6 then copies one into the other. (The source slide swaps the two in its initialisation line; step 6 disambiguates, and the version above is the consistent one.)

The ϵ-greedy selection in step 1 is the exploration story of Unit 11, unchanged by the presence of a network. It is still the case that a deterministic policy visits one action per state, and still the case that ϵ must start large and decay.

Example: the lunar lander

The example is the lunar lander from OpenAI Gymnasium. Four discrete actions are available: 0 do nothing, 1 fire the left orientation engine, 2 fire the main engine, 3 fire the right orientation engine.

The dynamics are a six-dimensional vector – coordinates (x, y, θ) and velocities $(v_x, v_y, \dot{\theta})$ – and the state carries two additional booleans recording whether each leg is in contact with the ground, so eight numbers in all.

The reward accumulates over the episode. At each step it is increased or decreased the closer or further the lander is to the landing pad; increased or decreased the slower or faster it is moving; decreased the more it is tilted away from horizontal; increased by 10 points for each leg in contact with the ground; decreased by 0.03 points per frame that a side engine fires and by 0.3 points per frame that the main engine fires. The episode receives an additional -100 or $+100$ for crashing or landing safely. An episode scoring at least 200 counts as solved. The lander starts at the top centre with a random initial force applied to its centre of mass, and the episode ends if it crashes, leaves the viewport, or comes to rest.

One convention note. Gymnasium hands out **rewards to be maximised**, while this course minimises a **cost**; the two differ by a sign, and the greedy action in the accompanying notebooks is accordingly an $\arg\max$ where every formula here is an $\arg\min$.

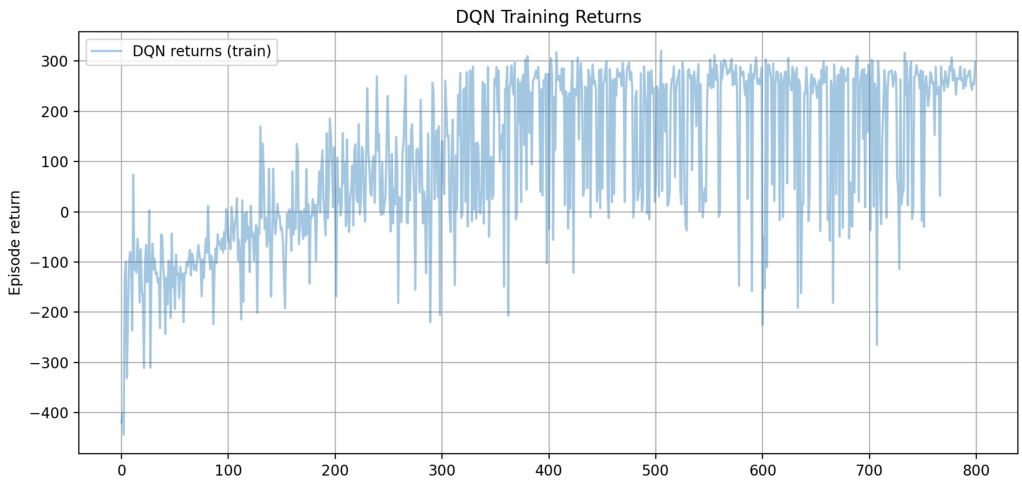


Fig. 10: Deep Q-learning on the lunar lander: the return of each training episode against episode number, over 800 episodes with a three-layer network. The trend is clearly upward and the scatter never disappears.

Two features of Fig. 10 are worth reading, because both are typical and neither is a defect.

The scatter never goes away. Even late in training, individual episodes land anywhere from −200 to +300. Part of that is the environment – the random initial push makes some episodes genuinely harder – and part is the residual exploration, where one random main-engine burn at the wrong moment costs an otherwise good landing. A learning curve that became a clean line would be the suspicious one.

Progress is not monotone. Stretches of improvement are interrupted by sharp drops. This is the price of bootstrapping on a moving approximation, and it is the visible form of the lost guarantee: nothing in this unit proves that the sequence of networks converges, and the plot is evidence that it works on this problem, not that it must.

Approximation inside policy iteration

The same substitution can be made, Fig. 11, inside the policy iteration of Unit 11, and doing so exposes a distinction that deep Q-learning hides.

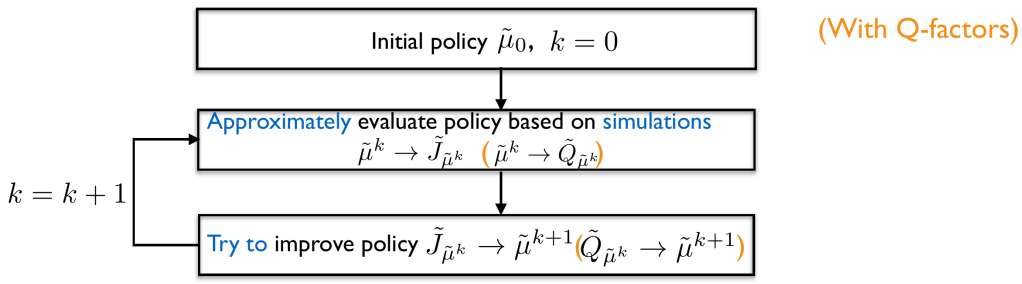


Fig. 11: Simulation-based policy iteration with Q-factors, as in Unit 11, with the approximation made explicit: both the evaluation and the improvement step now work with a parameterised $\tilde{J}_{\tilde{\mu}^k}(r_k)$ or $\tilde{Q}_{\tilde{\mu}^k}(r_k)$ rather than a table.

For problems with discrete states the only source of approximation is the use of simulation, which was Unit 11's subject. For stage decision problems we must additionally circumvent the curse of dimensionality by parameterising

$$\tilde{J}_{\tilde{\mu}^k}(r_k) \qquad (\tilde{Q}_{\tilde{\mu}^k}(r_k)),$$

and the improvement step then requires solving

$$u_k = \arg\min_{u_k \in U_k(x_k)} \mathbb{E}[g_k(x_k, u_k) + \tilde{J}_{\mu^{k+1}}(x_{k+1})] \qquad (u_k = \arg\min_{u_k \in U_k(x_k)} \tilde{Q}_{\mu^k}(x_k, u_k)),$$

where approximate methods such as a gradient method may be needed – the subject of the actor-critic methods of the next unit. For now the focus is the evaluation step.

An intermezzo on stochastic gradient methods

Stated once in general, since it is used twice below. For problems that involve the gradient of an expected value over some stochastic variable w,

$$\theta_{k+1} = \theta_k - \gamma \nabla_\theta \mathbb{E}_w[V(\theta, w)],$$

stochastic gradient methods simply replace the expectation by a single realization of w,

$$\theta_{k+1} = \theta_k - \gamma \nabla_\theta V(\theta, w).$$

When V is convex, and under the usual assumptions on γ – it must be time-varying and decaying – this converges to the optimal parameter.

Stochastic gradient with Monte-Carlo

Instead of keeping a table and updating it, as tabular TD and Monte-Carlo do, update the parameters of a network approximating the value function:

$$\theta_{k+1} = \theta_k - \gamma_k \nabla_\theta (J_i - \hat{J}(x_i; \theta))^2,$$

where J_i is **one Monte-Carlo return** obtained by starting in state x_i and running the policy to the end.

This is nothing more than stochastic gradient, in the exact sense of the intermezzo. The cost function it descends is

$$\min_\theta \ \mathbb{E}[(J(x) - \hat{J}(x; \theta))^2] = \sum_x \mu(x)(J(x) - \hat{J}(x; \theta))^2,$$

where $\mu(x)$ is the asymptotic probability of state x – the ratio of time spent in x under the policy being evaluated.

That weighting deserves a moment. The objective does not ask for a good fit everywhere; it asks for a good fit *where the policy goes*, weighted by how often it goes there. States the policy rarely visits contribute almost nothing and will be approximated badly. For evaluating a fixed policy that is exactly right. It becomes a problem only when the policy changes – the improvement step may send the system into states the previous policy avoided, where the approximation was never fitted. This is Unit 11's exploration problem, returning in the form the approximation gives it.

Semi-gradient with TD

The semi-gradient method is similar, but the sampled return of Monte-Carlo is replaced by the usual estimate of TD,

$$J_i \;\longrightarrow\; g(x_k, \mu(x_k)) + \hat{J}(x_{k+1}, \theta),$$

substituted into the same update. And here the substitution does something that the notation hides: the quantity replacing J_i **contains** θ.

This is **not** a stochastic gradient method, because we do not take the gradient of the error. We first take the gradient with respect to $\hat{J}(x_k, \theta)$ – treating the target as a constant – and only then replace the target by the TD return. Written out, the update is

$$\theta_{k+1} = \theta_k + \gamma_k \Big(g(x_k, \mu(x_k)) + \alpha \hat{J}(x_{k+1}; \theta_k) - \hat{J}(x_k; \theta_k) \Big) \nabla_\theta \hat{J}(x_k; \theta_k).$$

The name is apt: it is half a gradient. The θ inside the target is there, and is deliberately not differentiated.

Quick exercise: why not just take the whole gradient?

The semi-gradient update deliberately ignores the dependence of the target on θ. Differentiating through it instead would give a genuine gradient method on a genuine cost function, with all the convergence theory that brings. Why is that not what is done?

Solution

Because the cost function one would then be minimising is the wrong one, and minimising it gives the wrong answer.
Differentiating through the target means minimising the **mean squared Bellman error**,

$$\mathbb{E}\Big[\big(g(x_k, \mu(x_k)) + \alpha \hat{J}(x_{k+1}; \theta) - \hat{J}(x_k; \theta) \big)^2 \Big],$$

with respect to θ in both places. That is a well-defined objective and a real gradient method, and it has two problems.
The first is practical and is the famous one. The expectation is over the *random* successor x_{k+1}, and the expression is quadratic in it, so an unbiased estimate of the gradient needs **two independent samples** of the successor from the same state – the double sampling problem. With a simulator that can be reset to a given state one can obtain them; with a stream of experience from a real system one cannot, because the system only visits each state once and then moves on.
The second is more fundamental: the fixed point is wrong. The quantity we want is the solution of the projected Bellman equation – the best approximation, within the chosen family, of the value function – and the minimiser of the mean squared Bellman error is a different point. On examples one can construct, it is a noticeably worse approximation of J_μ than the semi-gradient fixed point, because it trades accuracy against making the Bellman residual small, and those are not the same goal.
So semi-gradient is not a lazy shortcut. It is a deliberate choice of which fixed point to converge to, and the price is that the update is not the gradient of anything, so its convergence has to be argued separately – which is possible for linear features, the next section, and not in general.

Linear features and the TD fixed point

For linear features the semi-gradient update can be analysed completely. When $\hat{J}(x, \theta) = \theta^\top \phi(x)$ with $\phi : \mathbb{R}^n \to \mathbb{R}^m$ and $m \ll n$, the update becomes

$$\theta_{k+1} = \theta_k + \gamma_k(J_i - \theta_k^\top \phi(x_k))\phi(x_k),$$

a linear iteration in the parameters. (The source slide prints this with a minus sign; the gradient of $(J-\theta^\top\phi)^2$ is $-2(J-\theta^\top\phi)\phi$, so descent adds the term.)

Being linear, it can be shown to converge, and the limit can be written down:

$$\theta^\star = \left(\tilde{\Phi}^\top\tilde{\Phi} - \alpha\tilde{\Phi}^\top\tilde{\Phi}'\right)^{-1}(\tilde{\Phi}^\top G),$$

where, over a collected trajectory of length T,

$$\begin{aligned}
\tilde{\Phi} &= \left[\phi(x_0), \phi(x_1), \dots, \phi(x_{T-1})\right]^\top, \\
\tilde{\Phi}' &= \left[\phi(x_1), \phi(x_2), \dots, \phi(x_T)\right]^\top, \\
G &= \left[g(x_0,u_0), g(x_1,u_1), \dots, g(x_{T-1},u_{T-1})\right]^\top.
\end{aligned}$$

The structure is worth reading. $\tilde{\Phi}$ holds the features of the states visited, $\tilde{\Phi}'$ the features of their successors, and G the costs incurred. The matrix being inverted compares a state's features against its successor's, discounted – which is the Bellman equation, expressed in the feature space.

LSTD

There is a second route to the same point, and it explains what the semi-gradient is converging to.

The value function of a policy solves $J_\mu = TJ_\mu$, where T is the Bellman operator for that policy. In general J_μ does not lie in the space of functions our architecture can represent, so the equation cannot be solved exactly. The fix is to **project** the right-hand side back into the feature space $\mathcal{J} = \{\Phi\omega \mid \omega \in \mathbb{R}^m\}$, with

$$\Phi = \begin{bmatrix} \phi_1(1) & \phi_2(1) & \dots & \phi_m(1) \\ \phi_1(2) & \phi_2(2) & \dots & \phi_m(2) \\ \dots & \dots & \dots & \dots \\ \phi_1(n) & \phi_2(n) & \dots & \phi_m(n) \end{bmatrix},$$

and solve the **projected Bellman equation** instead:

$$\tilde{J}_\mu = \arg\min_{J\in\mathcal{J}} \|J - \Pi TJ\|^2, \qquad \Pi TJ = \arg\min_{h\in\mathcal{J}} \|TJ - h\|^2.$$

A sampled version of this is the **Least Squares Temporal Difference** method, in which the weights solve

$$\theta = \arg\min_{\bar{\theta}\in\mathbb{R}^m} \sum_{k=1}^{S} \left\|g(x_k,\mu(x_k)) + \phi(x_{k+1})^\top\bar{\omega} - \phi(x_k)^\top\bar{\theta}\right\|^2, \qquad \omega = \arg\min_{\bar{\omega}} \sum_{k=1}^{S} \left\|\phi(x_k)^\top\bar{\omega} - \phi(x_k)^\top\theta\right\|^2.$$

Solving these leads to the same answer as the incremental iteration – called the **TD fixed point** –

$$\theta^\star = \left(\tilde{\Phi}^\top\tilde{\Phi} - \alpha\tilde{\Phi}^\top\tilde{\Phi}'\right)^{-1}(\tilde{\Phi}^\top G).$$

So the two methods agree, and the difference between them is purely computational: the semi-gradient iteration approaches the fixed point gradually as data arrives, while LSTD computes it in one solve from a stored batch. LSTD uses its data far more efficiently and needs no step size at all – there is no γ to tune – at the cost of an $m \times m$ solve and of requiring the architecture to be linear.

This is the one place in the unit where something exact survives, and the reason is the one given at the very beginning: linear in the parameters means a quadratic objective, and a quadratic objective has a closed form.

SARSA with function approximation

Finally, the same substitution applied to SARSA. In the same way we went from Q-learning to Q-learning with function approximation, we can go from SARSA to SARSA with function approximation.

SARSA with function approximation

Initialize the parameters θ of $Q(x, u, \theta)$.
For each episode:

A) Initialize x_0 and select $u_0 = \arg\min_u Q(x_0, u, \theta)$ with probability $1-\epsilon$, otherwise a random action.
B) For each time step k:

1. Execute u_k, observe the cost and the next state x_{k+1}.
2. Select $u_{k+1} = \arg\min_u Q(x_{k+1}, u, \theta)$ with probability $1-\epsilon$, otherwise a random action.
3. Compute the target:

$$z_k = \begin{cases} g(x_k, u_k) + g(x_{k+1}, \cdot), & \text{if } x_{k+1} \text{ is terminal} \\ g(x_k, u_k) + \alpha\, Q(x_{k+1}, u_{k+1}; \theta), & \text{otherwise.} \end{cases}$$

4. Perform a gradient descent step on the loss:

$$\theta \leftarrow \theta + \gamma(g(x_k, u_k) + \alpha\, Q(x_{k+1}, u_{k+1}; \theta) - Q(x_k, u_k; \theta))\nabla_\theta Q(x_k, u_k; \theta).$$

That final line is a semi-gradient step, in exactly the sense defined above: θ appears in the target and is not differentiated. Note also what is *absent* compared with deep Q-learning – there is no replay buffer and no target network. Neither omission is a simplification.

The replay buffer is impossible rather than merely unnecessary. A stored transition (x, u, g, x') is a complete training example for deep Q-learning, because its target maximises over the next action and does not care which policy produced the data. SARSA's target needs u', the action the policy took next, and a stored u' was chosen by an older policy. This is Unit 11's observation that a SARSA data set is a *sequence* rather than a *set*, restated as a property of the algorithm.

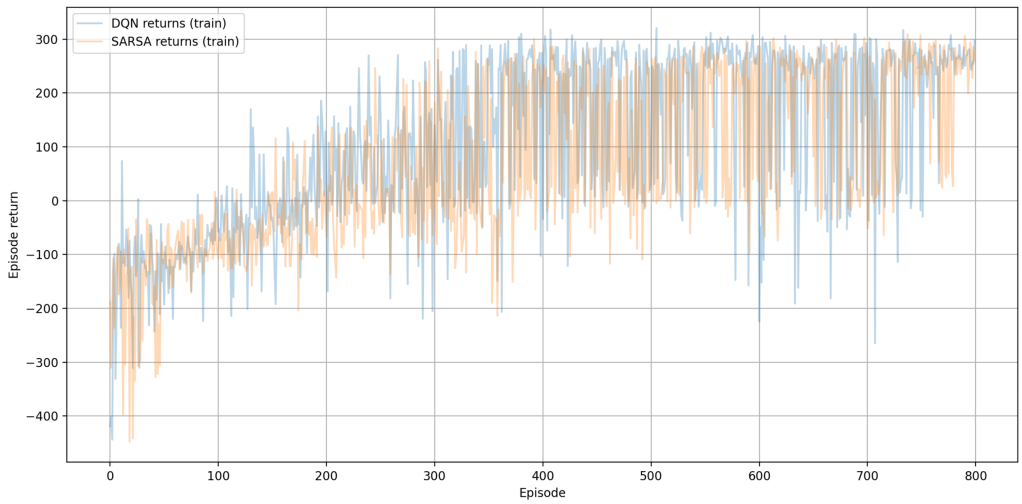


Fig. 12: Deep Q-learning and SARSA with function approximation on the lunar lander, 800 training episodes each. The two curves are essentially indistinguishable, which is the point.

Fig. 12 shows the comparison, and the result is the one Unit 11 found on the frozen lake: the two perform similarly. The reason is also the same, and is worth computing rather than asserting. With two or more

actions and an ϵ-greedy behaviour policy, the action SARSA bootstraps on coincides with the minimising action a fraction $1 - \epsilon + \epsilon/|U|$ of the time. With ϵ decayed to 0.02 and four actions that is 98.5% – so the two targets agree on all but one step in seventy, and the algorithms can hardly differ. Raising the exploration floor to 0.2 drops it to 85% and the two begin to separate. `A_12_LunarLanderDQNvsSARSA.ipynb` runs both.

Summary

The tabular methods of Units 10 and 11 need one entry per state, or per state-action pair. This unit replaces the table by a **parameterised approximation**, which is what makes continuous and very large state spaces tractable, and is the curse of dimensionality finally addressed.

Linear architectures $r^\top \phi(x)$ are linear in the parameters, not in the state, and can be very powerful if the features are right. Their great advantage is that training is a single least-squares solve with no local minima. Their great disadvantage is that somebody must choose the features.

Neural networks choose their own. A linear layer followed by a rectifier can build a pulse, pulses can build any continuous function, and deep networks learn features from the outputs of earlier layers. The price is a harder, non-convex optimisation problem.

Training an architecture is **regression**, and least-squares regression computes the conditional expectation $\mathbb{E}[J \mid x]$ – which is exactly what a cost-to-go is. **Stochastic gradient**, one sample at a time with back propagation computing the gradient, is what makes it affordable when the data set is large, which in reinforcement learning it always is.

Deep Q-learning is Unit 10's algorithm with the table replaced by a network, plus two devices that exist because the regression target now contains the parameters being fitted: an **experience replay** buffer, which breaks the correlation between consecutive samples, and a slowly updated **target network**, which stops the target from moving as fast as the estimate.

For policy evaluation with an approximation, a Monte-Carlo target gives a genuine **stochastic gradient**, while a TD target gives a **semi-gradient** – the parameters appear in the target and are deliberately not differentiated. This is a choice, not an oversight: differentiating through the target means minimising the Bellman residual, which needs two independent samples of each successor and converges to a different and generally worse fixed point.

With **linear features** the semi-gradient iteration is linear and converges to the **TD fixed point**, and **LSTD** computes that same point directly from a batch by solving the projected Bellman equation in a least-squares sense. Same answer, incrementally or in one solve.

What has been given up is worth repeating. Unit 10's convergence theorem was tabular, and rested on each update touching one entry. With an approximation, every update changes the prediction everywhere, the contraction argument fails, and none of the algorithms of this unit – deep Q-learning, semi-gradient TD, SARSA with function approximation – carries a guarantee. They are supported by the fact that they work. Linear features are the exception, and that is the reason to keep them in view.